Understanding the Spring Boot Backend

A beginner's guide to the Employee POC REST API (Spring Boot + Hibernate + H2)

This guide explains the Spring Boot / Hibernate / H2 concepts used in the **Employee POC** backend, written for a beginner. Everything is tied to the real files in your project under `backend/src/main/java/com/veltris/employee/`: `EmployeeApplication.java`, `model/Employee.java`, `repository/EmployeeRepository.java`, `service/EmployeeService.java`, `controller/EmployeeController.java`, `config/WebConfig.java`, the `exception/` classes, plus `resources/application.properties` and `pom.xml`.

1. The big picture: a layered application

A Spring Boot web app is organised in **layers**. A request enters at the top and travels down; the response travels back up. Each layer has one job.

```
    HTTP request (from the React app / Postman / curl)
                        |
                        v
   +-------------------------------------------------+
   |  Controller     EmployeeController.java         |  <- speaks HTTP
   |  (maps URLs + methods to Java methods)          |     (URLs, status codes, JSON)
   +-------------------------------------------------+
                        |  calls
                        v
   +-------------------------------------------------+
   |  Service        EmployeeService.java            |  <- business logic / rules
   |  (what to do, validation of existence, etc.)    |
   +-------------------------------------------------+
                        |  calls
                        v
   +-------------------------------------------------+
   |  Repository     EmployeeRepository.java         |  <- talks to the database
   |  (save / find / delete - no SQL written by you) |
   +-------------------------------------------------+
                        |  Hibernate turns calls into SQL
                        v
   +-------------------------------------------------+
   |  Database       H2 (in-memory)                  |  <- stores the rows
   +-------------------------------------------------+
```

Why split it up? This is **separation of concerns**: the controller never touches the database, the repository never thinks about HTTP. Each piece is small, testable, and easy to change.

2. Spring Boot & the entry point

`EmployeeApplication.java` starts everything:

```
@SpringBootApplication
public class EmployeeApplication {
   public static void main(String[] args) {
       SpringApplication.run(EmployeeApplication.class, args);
   }
}
```

`@SpringBootApplication` is one annotation that does three big things:

- **Component scanning** — Spring looks through this package (and sub-packages like `controller`, `service`, `repository`) and picks up your classes.
- **Auto-configuration** — seeing an H2 driver and JPA on the classpath, Spring automatically sets up a database connection, Hibernate, an embedded Tomcat web server, etc.
- **Configuration** — it marks this class as a source of settings.

`SpringApplication.run(...)` boots the app: it starts the web server on port 8080 and builds the **application context** (explained next).

3. The core idea: Dependency Injection (DI)

This is the concept that makes Spring "Spring." Normally in Java you create objects yourself with `new`. In Spring, **you don't** — Spring creates them for you and hands them to whoever needs them. Those Spring-managed objects are called **beans**, and they live in a container called the **application context**.

Look at how `EmployeeService` gets its repository:

```
@Service
public class EmployeeService {
   private final EmployeeRepository employeeRepository;

   // Spring sees this constructor and automatically passes in the repository bean.
   public EmployeeService(EmployeeRepository employeeRepository) {
       this.employeeRepository = employeeRepository;
   }
}
```

You never wrote `new EmployeeRepository()` anywhere. Spring did it and **injected** it through the constructor. This is **constructor injection**.

```
  Spring's application context (a box of ready-made objects/"beans")
 +-------------------------------------------------------------+
 |   EmployeeRepository  --->  EmployeeService  --->  EmployeeController
 |        (bean)      injected     (bean)      injected    (bean)  |
 +-------------------------------------------------------------+
      Spring builds each one and wires them together at startup.
```

Why is this good? Your classes just declare what they need; Spring supplies it. That makes code loosely coupled and easy to test (you can pass in a fake repository in a test).

> The annotations `@RestController`, `@Service`, and `@Repository` all tell Spring "this class is a bean — manage it." They are the same idea (`@Component`) with role-specific names that document intent.

4. The Entity: mapping a Java class to a database table (JPA + Hibernate)

`Employee.java` is a plain Java class turned into a database table by **JPA** annotations. **JPA** is the standard (the rulebook); **Hibernate** is the engine that implements it and writes the actual SQL. This mapping is called **ORM** (Object-Relational Mapping).

```
@Entity
@Table(name = "employees")
public class Employee {

   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;

   @NotBlank
   @Column(nullable = false)
   private String firstName;

   @Email
   @Column(nullable = false, unique = true)
   private String email;

   @PositiveOrZero
   private Double salary;
   // ... getters and setters ...
}
```

```
  Java object (Employee)                Database table (EMPLOYEES)
 +----------------------+              +----+-----------+------------------+--------+
 | id        = 1        |   maps to    | ID | FIRST_NAME | EMAIL           | SALARY |
 | firstName = "Ada"    | -----------> +----+-----------+------------------+--------+
 | email     = "a@x.com"|              |  1 | Ada        | a@x.com         | 120000 |
 | salary    = 120000   |              +----+-----------+------------------+--------+
 +----------------------+
```

- `@Entity` — "this class is a table." `@Table(name="employees")` names it.
- `@Id` — the primary key. `@GeneratedValue(IDENTITY)` lets the DB auto-generate the id (1, 2, 3, ...), so you never set it.
- `@Column` — maps a field to a column; `nullable=false` and `unique=true` become real database constraints.
- `@NotBlank`, `@Email`, `@PositiveOrZero` — **validation** rules (covered in section 8).

Because Hibernate knows this mapping, it can generate `CREATE TABLE`, `INSERT`, `SELECT`, `UPDATE`, and `DELETE` for you — you write Java, not SQL.

5. The Repository: free CRUD with Spring Data JPA

Here is the whole repository:

```
@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {
}
```

That is it — an empty **interface**, no implementation. By extending `JpaRepository<Employee, Long>` (the entity type, and the id type), you instantly get a set of ready-made methods:

```
 JpaRepository<Employee, Long>  gives you, for free:
+-------------------------------------------------------------+
|  findAll()          -> SELECT * FROM employees              |
|  findById(id)       -> SELECT ... WHERE id = ?              |
|  save(employee)     -> INSERT (new) or UPDATE (existing)    |
|  deleteById(id)     -> DELETE FROM employees WHERE id = ?   |
|  count(), existsById(id), ... and more                      |
+-------------------------------------------------------------+
```

At startup Spring Data **generates the implementation** of this interface behind the scenes and registers it as a bean. You just declare the interface and call the methods.

6. The Service: business logic

`EmployeeService.java` sits between the controller and the repository. It holds the "what should happen" rules. Example — fetching one employee or failing clearly:

```
public Employee getEmployeeById(Long id) {
   return employeeRepository.findById(id)
       .orElseThrow(() -> new ResourceNotFoundException("Employee not found with id: " + id));
}

public Employee updateEmployee(Long id, Employee details) {
   Employee employee = getEmployeeById(id);   // reuse: 404 if it doesn't exist
   employee.setFirstName(details.getFirstName());
   employee.setEmail(details.getEmail());
   // ... copy the other fields ...
   return employeeRepository.save(employee);   // save() updates because id is set
}
```

- `findById` returns an `Optional<Employee>` (maybe-there, maybe-not). `orElseThrow` converts "not there" into a clear exception.
- `save()` is smart: with no id it **inserts**; with an existing id it **updates**. That is why update loads the row first, copies fields, then saves.
- Keeping this logic out of the controller means you could reuse it from a scheduled job, a different API, or a test without dragging HTTP along.

7. The Controller: turning HTTP into method calls (REST)

`EmployeeController.java` is the front door. **REST** is a convention: use HTTP *methods* (GET/POST/PUT/DELETE) on *resource URLs* (`/api/employees`) to mean read/create/update/delete.

```
@RestController
@RequestMapping("/api/employees")     // every URL below starts with this
public class EmployeeController {

   private final EmployeeService employeeService;   // injected

   @GetMapping                                        // GET /api/employees
   public ResponseEntity<List<Employee>> getAll() {
       return ResponseEntity.ok(employeeService.getAllEmployees());
   }

   @PostMapping                                       // POST /api/employees
   public ResponseEntity<Employee> create(@Valid @RequestBody Employee employee) {
       Employee saved = employeeService.createEmployee(employee);
       return new ResponseEntity<>(saved, HttpStatus.CREATED);   // 201
   }

   @PutMapping("/{id}")                               // PUT /api/employees/5
   public ResponseEntity<Employee> update(@PathVariable Long id,
                                          @Valid @RequestBody Employee employee) {
       return ResponseEntity.ok(employeeService.updateEmployee(id, employee));
   }
}
```

The key annotations:

Annotation	What it does
@RestController	This class handles web requests and returns data (JSON), not HTML pages.
@RequestMapping("/api/employees")	The base URL shared by every method in the class.
@GetMapping / @PostMapping / @PutMapping / @DeleteMapping	Map an HTTP method (+ optional sub-path) to one Java method.
@PathVariable Long id	Pulls the `{id}` out of the URL into the method argument.
@RequestBody Employee	Converts the incoming JSON body into an Employee object automatically.
ResponseEntity<...>	Lets you set both the response body **and** the HTTP status code (200, 201, 204...).

So the five endpoints map cleanly to the five operations:

```
  GET    /api/employees        -> list all      -> 200 OK
GET    /api/employees/{id}   -> get one       -> 200 OK   (or 404)
POST   /api/employees        -> create        -> 201 Created
PUT    /api/employees/{id}   -> update        -> 200 OK   (or 404)
DELETE /api/employees/{id}   -> delete        -> 204 No Content
```

8. Validation: rejecting bad data

On create/update the controller writes `@Valid @RequestBody Employee`. The `@Valid` tells Spring: before running my method, check the rules declared on the `Employee` fields (`@NotBlank`, `@Email`, `@PositiveOrZero`). If any fail, the method never runs.

```
   JSON body arrives
       |
       v
  @Valid checks the @NotBlank/@Email/@PositiveOrZero rules
       |
       +-- all pass --> controller method runs --> 200 / 201
       |
       +-- some fail --> Spring throws MethodArgumentNotValidException
                              |
                              v
                  GlobalExceptionHandler -> 400 Bad Request + field messages
```

9. Exception handling: clean errors in one place

Rather than `try/catch` in every method, `GlobalExceptionHandler.java` handles errors centrally with `@RestControllerAdvice`:

```
@RestControllerAdvice
public class GlobalExceptionHandler {

   @ExceptionHandler(ResourceNotFoundException.class)
   public ResponseEntity<...> handleNotFound(ResourceNotFoundException ex) {
       // build a tidy JSON body with status 404
   }

   @ExceptionHandler(MethodArgumentNotValidException.class)
   public ResponseEntity<...> handleValidation(...) {
       // collect per-field messages, return status 400
   }
}
```

- `@RestControllerAdvice` — "watch every controller; if these exceptions escape, I will handle them."
- `ResourceNotFoundException` (thrown by the service) becomes a clean **404** with a message.
- A validation failure becomes a **400** listing which fields were wrong — which is exactly what your React app reads to show the red error banner.

10. The database & configuration (H2 + application.properties)

`application.properties` configures the app without touching Java code:

```
server.port=8080

spring.datasource.url=jdbc:h2:mem:employeedb   # in-memory H2 database
spring.datasource.username=sa

spring.jpa.hibernate.ddl-auto=update           # let Hibernate create/adjust tables
spring.jpa.show-sql=true                        # print the SQL it runs

spring.h2.console.enabled=true                  # browser DB viewer at /h2-console
```

- **H2** is a tiny database that runs *inside* your app. `mem:` means **in-memory** — fast, zero setup, but data resets when the app stops (fine for a POC).
- `ddl-auto=update` — Hibernate reads your `@Entity` classes and creates/updates the tables automatically at startup, so you never write `CREATE TABLE`.
- `/h2-console` — a web page to run SQL against the live database (JDBC URL `jdbc:h2:mem:employeedb`, user `sa`, blank password).

11. CORS: letting the React app call the API

Browsers block a page on `localhost:5173` (React) from calling a server on `localhost:8080` (Spring Boot) unless the server explicitly allows it. That rule is the **Same-Origin Policy**; the permission is **CORS**. `WebConfig.java` grants it:

```
@Configuration
public class WebConfig implements WebMvcConfigurer {
   @Override
   public void addCorsMappings(CorsRegistry registry) {
       registry.addMapping("/api/**")
               .allowedOrigins("http://localhost:5173")   // the React dev server
               .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS");
   }
}
```

Without this, the browser would refuse the requests and your table would stay empty even though the API works in Postman.

12. pom.xml: dependencies & the Maven build

`pom.xml` is Maven's project file. It lists **starters** — curated bundles of libraries — so you don't hunt down individual jars or their versions:

```
spring-boot-starter-web         -> REST controllers + embedded Tomcat web server
spring-boot-starter-data-jpa    -> Spring Data JPA + Hibernate
spring-boot-starter-validation  -> the @NotBlank / @Email rules
h2                              -> the H2 database (runtime only)
```

- The `spring-boot-starter-parent` at the top sets sensible versions for everything, so the dependencies list stays clean (no version numbers to manage).
- The **Maven wrapper** (`mvnw` / `mvnw.cmd`) means you don't need Maven pre-installed — the wrapper downloads the right Maven automatically. `mvnw.cmd spring-boot:run` compiles and starts the app.

13. End-to-end: what happens on "Add Employee"

Tying every layer together for a single `POST` from the React form:

```
1. React sends   POST /api/employees   with JSON body
                       |
                       v
2. Tomcat (embedded) hands it to Spring, which routes it by URL+method
                       |
                       v
3. EmployeeController.create(...)
    - @RequestBody turns JSON --> Employee object
    - @Valid checks the field rules  (fail -> 400 via GlobalExceptionHandler)
                       |
                       v
4. EmployeeService.createEmployee(employee)   (business logic)
                       |
                       v
5. EmployeeRepository.save(employee)
                       |
                       v
6. Hibernate generates:  INSERT INTO employees (...) VALUES (...)
```

```
                        |
                        v
7. H2 stores the row and returns the generated id (e.g. 1)
                        |
                        v
8. Controller returns ResponseEntity(saved, 201 CREATED)
                        |
                        v
9. Spring serialises the Employee --> JSON --> back to React
       React refreshes its list; the new row appears in the table.
```

The five ideas to remember:

1. **Layers**: Controller (HTTP) → Service (logic) → Repository (data) → DB.
2. **Dependency Injection**: Spring creates objects (beans) and wires them for you.
3. **Annotations** configure behaviour: `@Entity`, `@RestController`, `@Service`, `@GetMapping`, `@Valid`.
4. **JPA/Hibernate** maps objects to tables and writes the SQL; Spring Data gives CRUD for free.
5. **ResponseEntity + exception handling** give clean HTTP status codes (200/201/204/400/404).

14. Quick glossary

Term	Meaning
Spring Boot	Framework that auto-configures a Java web app so you write mostly business code.
Bean	An object Spring creates and manages for you.
Application context	Spring's container holding all the beans.
Dependency Injection	Spring supplies the objects a class needs (usually via its constructor).
Annotation	A @Tag on a class/method/field that tells Spring how to treat it.
JPA	The Java standard for mapping objects to database tables.
Hibernate	The engine implementing JPA; it generates the SQL.
ORM	Object-Relational Mapping - objects <-> table rows.
Repository	Interface giving CRUD database methods (Spring Data JPA implements it).
Entity	A class mapped to a database table (@Entity).
REST	Using HTTP methods on resource URLs to mean read/create/update/delete.

ResponseEntity	An object letting you set the HTTP status code and body.
CORS	Server permission allowing a browser page from another origin to call the API.
H2	A lightweight database; here it runs in-memory inside the app.
Maven / pom.xml	Build tool and its project file listing dependencies (starters).